

A black and white photograph of a snake, likely a cobra, coiled around a rough-textured log. The snake's head is raised, and its tongue is flicking out. The scales on its body are highly detailed and catch the light. The background is dark and out of focus.

PYTHON

Getting Started with Python

About Python

Development started in the 1980's by Guido van Rossum.

- Only became popular in the last decade or so.

Python 2.x currently dominates, but Python 3.x is the future of Python.

- Interpreted, very-high-level programming language.
- Supports a multitude of programming paradigms.
 - OOP, functional, procedural, logic, structured, etc.

General purpose.

- Very comprehensive standard library includes numeric modules, crypto services, OS interfaces, networking modules, GUI support, development tools, etc.

Philosophy: From *The Zen of Python*

Beautiful is better than ugly

Explicit is better than implicit.

Simple is better than complex.

Complex is better than complicated.

Flat is better than nested.

Sparse is better than dense.

Readability counts.

Special cases aren't special enough to break the rules.

Although practicality beats purity.

Errors should never pass silently.

Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.

There should be one-- and preferably only one -- obvious way to do it. Although that way may not be obvious at first unless you're Dutch.

Now is better than never. Although never is often better than *right* now.

If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.

Namespaces are one honking great idea -- let's do more of those!

Notable Features

Easy to learn.

Supports quick development.

Cross-platform.

Open Source.

Extensible.

Embeddable.

Large standard library and active community.

Useful for a wide variety of applications.

Interpreter

- The standard implementation of Python is interpreted.
- The interpreter translates Python code into bytecode, and this bytecode is executed by the Python VM (similar to Java).
- Two modes: normal and interactive.
 - Normal mode: entire .py files are provided to the interpreter.
 - Interactive mode: read-eval-print loop (REPL) executes statements piecewise.

Interpreter: Normal mode

- Let's write our first Python program!
- In our favorite editor, let's create helloworld.py with the following contents:
- From the terminal:

```
print ("Hello, World!")
```

```
$ python helloworld.py  
Hello, World!
```

Note: In Python 2.x, print is a statement. In

Python 3.x, it is a function.

If you want to get into the 3.x habit, include at the beginning:

```
from __future__ import print_function
```

Now, you can write

```
print("Hello, World! ")
```

Interpreter: Interactive mode

- Let's accomplish the same task (and more) in interactive mode.
- Some options:
 - c : executes single command.
 - O: use basic optimizations.
 - d: debugging info.

```
$ python
>>> print ("Hello, World!")

>>> hellostring = "Hello, World!"
>>> hellostring

>>> 2*5

>>> 2*hellostring

>>> for i in range(0,3):
...     print ("Hello, World!")
...

>>> exit()
$
```

Some fundamentals

- Whitespace is significant in Python. Where other languages may use {} or (), Python uses indentation to denote code blocks.

```
# here's a comment
for i in range(0,3):
    print (i)
def myfunc():
    """here's a comment about
    the myfunc function"""
    print ("I'm in a
function!")
```

- Comments
 - Single-line comments denoted by #.
 - Multi-line comments begin and end with three “s.
 - Typically, multi-line comments are meant for documentation.
 - Comments should express information that cannot be expressed in code – do not restate code.

Python comments

In Python, there are several types of comments. They are described below:

- **Single-Line Comments**

`# This code is to show an example of a single-line comment`

- **Multi-Line Comments:** Python does not provide the facility for multi-line comments. However, there are indeed many ways to create multi-line comments

Using multipli #

`"Questo è un commento"`

`"""This is`

`A comment"""`

Python typing

- Python is a strongly, dynamically typed language.
- Strong Typing
 - Obviously, Python isn't performing static type checking, but it does prevent mixing operations between mismatched types.
 - Explicit conversions are required in order to mix types.
 - Example: `2 + "four"` ← not going to fly
- Dynamic Typing
 - All type checking is done at runtime.
 - No need to declare a variable or give it a type before use.

Let's start by looking at Python's built-in data types.

Variable & Assignment Statements

- A variable is a name that refers to a value.
- Variables let us store and reuse values in several places.
- But to do this we need to define the variable, and then tell it to refer to a value.

~~int Variable~~

- We do this using an assignment statement

Variable = expression

Variable is a reference

- In python everything is considered as an object.
- In python we do not have variables instead python has names.
- When we declare a name/variable in python it stores the name in the memory.
- Every python name/variable points to its **reference** in the memory and reference points to the object.

`a = 100`

"a" is name and it's points to the reference "31365024"

the reference points to the object "100" which is in binary format.

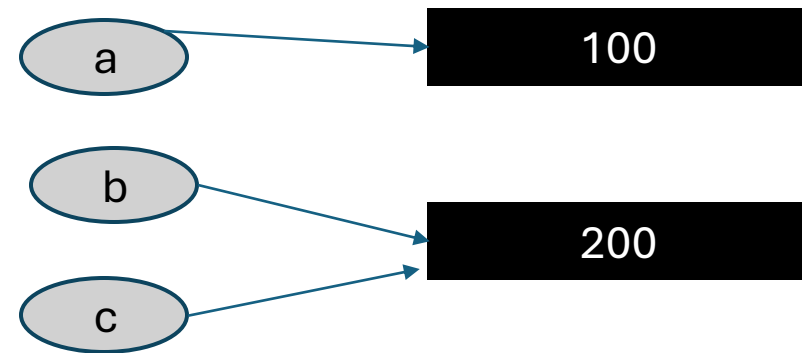
A name is just a label for an object.

In python each object can have lots of names.

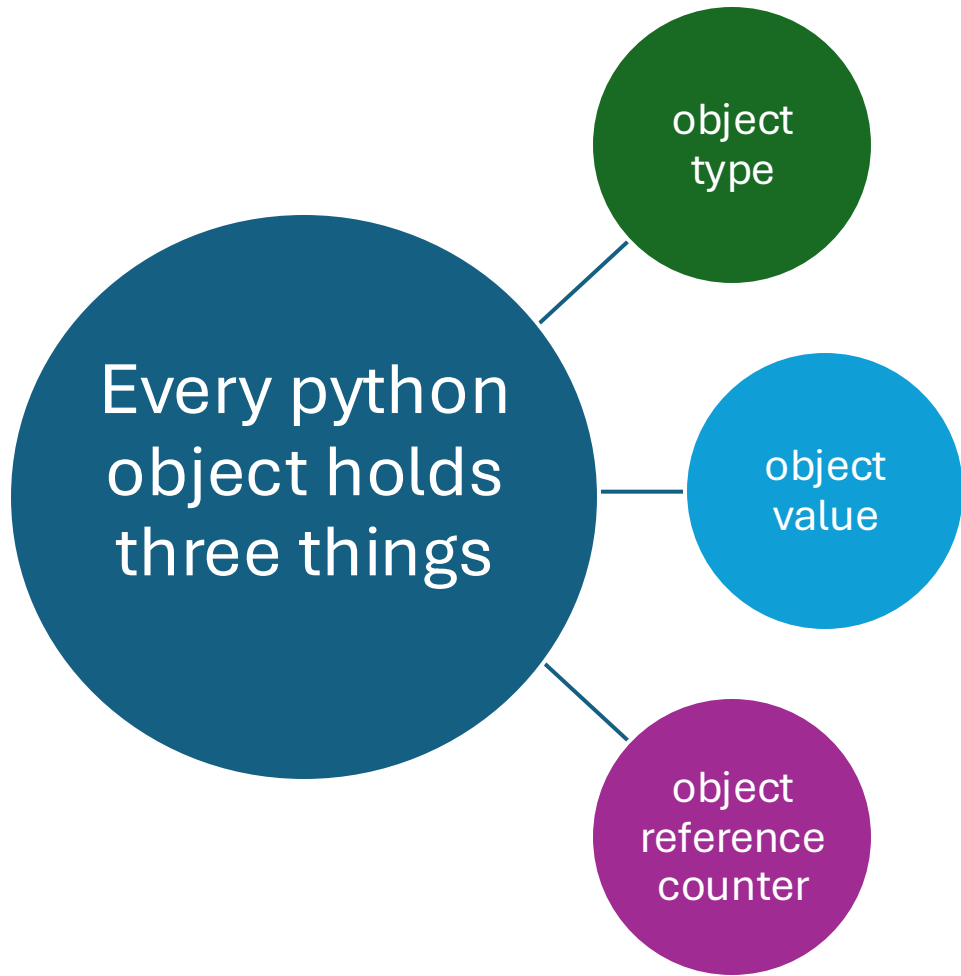
Stack and heap

- Stack memory stores the state of the program.
- Private heap memory stores the objects and data structures.
- Python memory manager will takes care of the allocation of the *private heap memory* to the names/variables.
- Python manager uses a Reference counting, Garbace collector

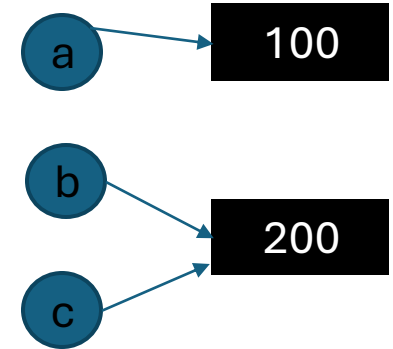
```
a = 100  
b = 200  
c = 200
```

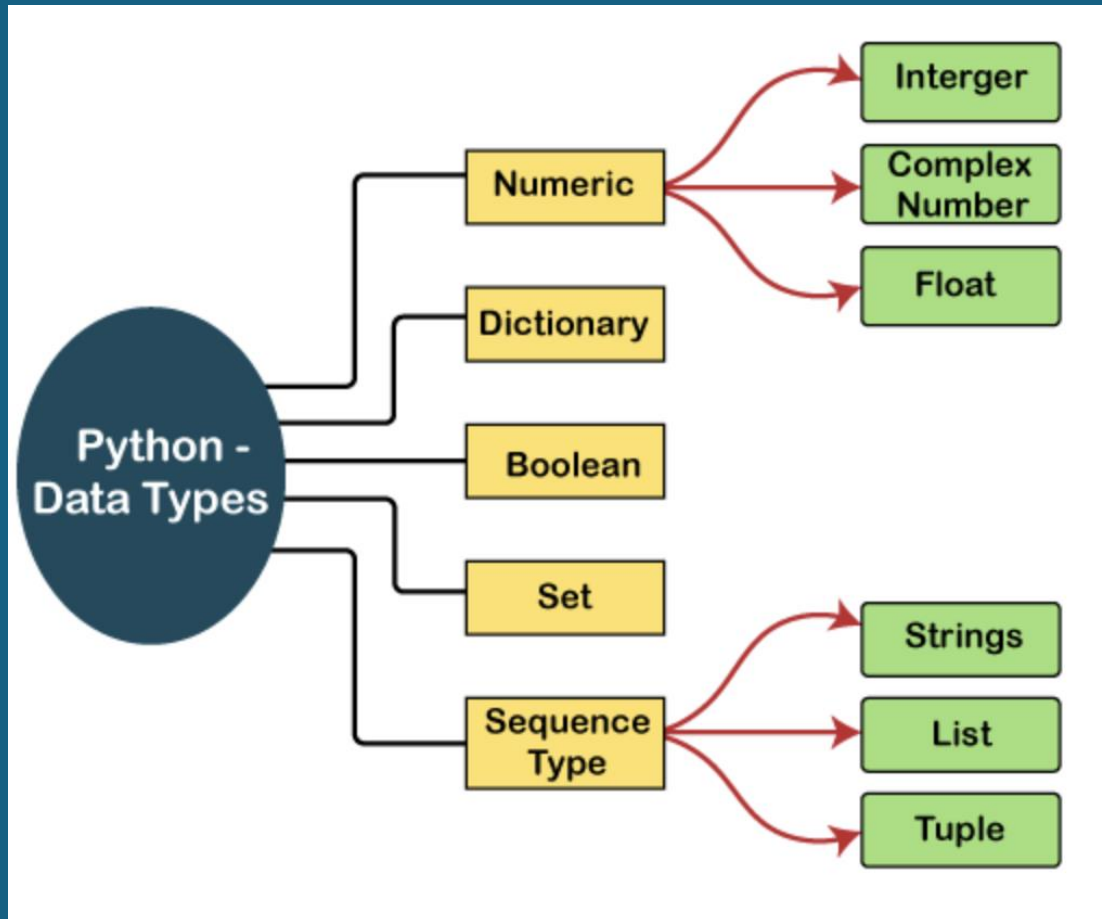


Python object



```
a = 100  
b = 200  
c = 200
```





PYTHON STANDARD TYPES

Numeric Types: Number

The subtypes are int, long, float and complex.

- Their respective constructors are int(), long(), float(), and complex().
- All numeric types, except complex, support the typical numeric operations you'd expect to find
- Mixed arithmetic is supported, with the “narrower” type widened to that of the other. The same rule is used for mixed comparisons.

Numeric Types

- Numeric
 - **int**: equivalent to C's long int in 2.x but unlimited in 3.x.
 - **float**: equivalent to C's doubles.
 - **long**: unlimited in 2.x and unavailable in 3.x.
 - **complex**: complex numbers.
- Supported operations include constructors (i.e. `int(3)`), arithmetic, negation, modulus, absolute value, exponentiation, etc.

```
$ python
>>> 3 + 2
5
>>> 18 % 5
3
>>> abs(-7)
7
>>> float(9)
9.0
>>> int(5.3)
5
>>> complex(1,2)
(1+2j)
>>> 2 ** 8
256
```

Size of int Tipi interi

Dynamic Integer Management

- Unlike languages like C or Java, Python 3 does not use fixed-size integers. Here's what that means:
- No Fixed Size Python's int type can grow as needed to represent very large numbers.
- Memory Usage Depends on Value The larger the number, the more memory Python allocates to store it.
- Internally Flexible Structure Python uses a dynamic internal representation that automatically expands to handle arbitrarily large integers.

Benefits

- No overflow errors for large integers
- Simplifies working with big numbers (e.g. cryptography, scientific computing)

```
x = 10  
y = 123456789012345678901234567890
```

Int dimension

	Dimensione (byte)
	28
	28
100	28
10_000	28
10**10	32
10**50	48
10**100	72
10**500	248

Arithmetic Operators in Python

Operator	Description
+ (Addition)	Adds two operands together.
– (Subtraction)	Subtracts the second operand from the first.
/ (Division)	Returns the quotient of dividing the first operand by the second. Example: $a = 20, b = 10 \rightarrow a / b = 2.0$
* (Multiplication)	Multiplies one operand by the other.
% (Remainder)	Returns the remainder of dividing the first operand by the second. Example: $a = 20, b = 10 \rightarrow a \% b = 0$
** (Exponentiation)	Raises the first operand to the power of the second. Example: $2 ** 3 = 8$
// (Floor Division)	Returns the integer part (floor) of the division result. Example: $7 // 2 = 3$

Operator	Meaning	Example
<code>==</code>	Equal to	<code>a == b</code> returns True if a equals b
<code>!=</code>	Not equal to	<code>a != b</code> returns True if a is not equal to b
<code>></code>	Greater than	<code>a > b</code> returns True if a is greater than b
<code><</code>	Less than	<code>a < b</code> returns True if a is less than b
<code>>=</code>	Greater than or equal to	<code>a >= b</code> returns True if a is greater than or equal to b
<code><=</code>	Less than or equal to	<code>a <= b</code> returns True if a is less than or equal to b

Operator	Meaning	Example
<code>+=</code>	Add and assign	<code>a += b</code> → <code>a = a + b</code>
<code>-=</code>	Subtract and assign	<code>a -= b</code> → <code>a = a - b</code>
<code>*=</code>	Multiply and assign	<code>a *= b</code> → <code>a = a * b</code>
<code>/=</code>	Divide and assign	<code>a /= b</code> → <code>a = a / b</code>
<code>//=</code>	Floor divide and assign	<code>a //= b</code> → <code>a = a // b</code>
<code>%=</code>	Modulo and assign	<code>a %= b</code> → <code>a = a % b</code>
<code>**=</code>	Exponentiate and assign	<code>a **= b</code> → <code>a = a ** b</code>

Funzioni Built_in su interi:

Function	Description	Example
<code>abs(x)</code>	Returns the absolute value of <code>x</code>	<code>abs(-5) → 5</code>
<code>pow(x, y)</code>	Returns <code>x</code> raised to the power of <code>y</code>	<code>pow(2, 3) → 8</code>
<code>round(x)</code>	Rounds a number to the nearest integer	<code>round(4.6) → 5</code>
<code>divmod(a, b)</code>	Returns a tuple (<code>a // b</code> , <code>a % b</code>)	<code>divmod(7, 3) → (2, 1)</code>
<code>max(iterable)</code>	Returns the largest item	<code>max([3, 7, 2]) → 7</code>
<code>min(iterable)</code>	Returns the smallest item	<code>min([3, 7, 2]) → 2</code>
<code>sum(iterable)</code>	Returns the sum of all items	<code>sum([1, 2, 3]) → 6</code>
<code>bin(x)</code>	Converts an integer to binary string	<code>bin(10) → '0b1010'</code>
<code>hex(x)</code>	Converts an integer to hexadecimal string	<code>hex(255) → '0xff'</code>
<code>oct(x)</code>	Converts an integer to octal string	<code>oct(8) → '0o10'</code>
<code>int(x)</code>	Converts a value to an integer	<code>int("42") → 42</code>



Type Float



Float

In Python, the float type represents a decimal number (that is, with a fractional part, although Python uses a dot instead of a comma).

A float is a double-precision number, according to the IEEE 754 standard. It is used to represent real values.

What is its size?

- A float in Python occupies 64 bits (that is, 8 bytes).
- This allows for a precision of about 15–17 decimal digits.
- The range of representable values is very large:
 - From approximately $\pm 2.2 \times 10^{-308}$ a $\pm 1.8 \times 10^{308}$



```
x = 3.14
```

```
y = 345.78
```


Built-in Functions for Floats

- These are general Python functions that work with floats:
- **float(x)**
Converts a number or string x to a floating-point number.
- **round(x, n)**
Rounds x to n decimal places (default is 0).
- **abs(x)**
Returns the absolute value of x.
- **pow(x, y)**
Returns x raised to the power y.
- **sum(iterable)**
Sums all elements in an iterable (floats included).
- **min(iterable) / max(iterable)**
Returns the smallest or largest value.

Float Methods

- Every float object has these methods:
- **float.as_integer_ratio()**
Returns a tuple (numerator, denominator) representing the float as a fraction.
- **float.is_integer()**
Returns True if the float is an integer value (e.g., 3.0).
- **float.hex()**
Returns a hexadecimal string representation of the float.
- **float.fromhex(s)** (*class method*)
Creates a float from a hexadecimal string.

+
•
○

TYPE Complex

+
•
○

Complex variable

```
z1 = 2 + 3j
```

```
z2 = 1 - 1j
```

```
somma = z1 + z2
```

```
differenza = z1 - z2
```

```
prodotto = z1 * z2
```

```
quoziente = z1 / z2
```

```
print(somma)
```

```
print(differenza)
```

```
print(prodotto)
```

```
print(quoziente)
```

TYPE boolean

+
•
○

Tipo boolean

In **Python**, the **bool** (boolean) type represents a **logical value**: it can only be **True** or **False**.



What is a boolean?

It is used for:

- Controlling conditions (if, while, etc.)
- Representing the result of comparisons (==, >, <, etc.)
- Handling boolean logic (and, or, not)

Even though a boolean represents only two states, in Python:

- A **bool** variable occupies **28 bytes**.
- This includes the **overhead of the Python object**, not just the logical value.

and, or , not

```
>>> True or False
True
>>> True and False
False
>>> not True
False
>>> not False
True
```

```
>>> 3 and 5
5
>>> 0 and 5
0
>>> True and 5
5
>>> False and 5
False
```

```
>>> 3 or 5
3
>>> 0 or 5
5
>>> True or 5
True
>>> False or 5
5
```

built_in function

bool(x)

- bool(0) → False,
- bool("ciao") → True
- bool("0") → True
- bool(0) → False
- int(True) → 1
- int(False) → 0

Altre funzioni

La funzione `type()` mi restituisce il tipo della variabile.
`type` : identificatore -> tipo

```
a = 5
print type(a)

b = 40.5
print(type(b))

c = 1+3j
print(type(c))
```

```
<class 'int'>
<class 'float'>
<class 'complex'>
```


casting

- In Python, **casting** (also called **type conversion**) means converting a value from one data type to another. There are two main types:

Implicit Casting (Type Promotion)

- Python automatically converts one type to another when needed (usually from smaller to larger types):

```
x = 5          # int
y = 2.0        # float
result = x + y  # int + float → float
print(result)  # 7.0
```

Explicit Casting (Manual Conversion)

- You use built-in functions to convert types:
- **int(x)** → Converts x to an integer (truncates decimals).
- **float(x)** → Converts x to a floating-point number.
- **str(x)** → Converts x to a string.
- **bool(x)** → Converts x to a boolean (False if x is 0, empty, or None; otherwise True).

```
a = "10"  
b = int(a)           # '10' → 10  
c = float(b)         # 10 → 10.0  
d = str(c)           # 10.0 → '10.0'  
e = bool(d)          # '10.0' → True
```

Special case

- Converting float to int **truncates** (does not round):

```
print(int(3.99)) # 3
```

Converting non-numeric strings to int or float raises an error:

```
int("abc") # ValueError
```

Dynamic typing

- In Python, you do not explicitly declare the type of a variable.
- The interpreter determines the type of a variable at **runtime** based on the assigned value.
- The type associated with a variable can change during execution.
- This is called **dynamic typing**.
- The built-in function `type()` returns the class of a variable.

```
>>> x = "Pippo"; print(type(x))  
<class, 'str'>  
>>> x = 123; print(type(x))  
<class, 'int'>  
>>> x = 3.14; print(type(x))  
<class, 'float'>
```

Function type()

- **type()** function to know the data-type of the variable.
- **isinstance()** function is used to check an object belongs to a particular class.

```
a = 5
print("The type of a", type(a))

b = 40.5
print("The type of b", type(b))

c = 1+3j
print("The type of c", type(c))
print(" c is a complex number", isinstance(1+3j,complex))
```

```
The type of a <class 'int'>
The type of b <class 'float'>
The type of c <class 'complex'>
c is complex number: True
```

function type

```
from __future__ import print_function
```

```
def funzione ():  
    return 2,4,5,7
```

```
x = funzione()  
print(type(x))
```

```
from __future__ import print_function
```

```
def funzione ():  
    return 2
```

```
x = funzione()  
print(type(x))
```



Type sequence

Sequence data types

- There are seven sequence subtypes: **strings**, **Unicode strings**, **lists**, **tuples**, **bytearrays**, **buffers**, and **range** objects.
- All data types support arrays of objects but with varying limitations.
- The most commonly used sequence data types are **strings**, **lists**, and **tuples**.
- The **range** data type finds common use in the construction of enumeration-controlled loops.
- The others are used less commonly.

Shared Characteristics

- Indexing Access elements using zero-based indices: `seq[0]`, `seq[1]`, etc.
- Slicing Extract sub-sequences: `seq[start:end]`
- Iteration Use loops to traverse elements: `for item in seq`
- Length Get number of elements: `len(seq)`
- Membership Testing Check if an item exists: `item in seq`
- Concatenation Combine sequences: `seq1 + seq2`
- Repetition Repeat elements: `seq * n`
- Immutability or immutable
- Built-in Functions Support Functions like

Python instruction



Simple instruction

. pass

- **What it does:** Does nothing. It's a placeholder when a statement is required syntactically but you don't want any action.

continue

- **What it does:** Skips the rest of the current loop iteration and moves to the next iteration.

break

- **What it does:** Exits the loop immediately.

return (*in functions*)

- Ends the function and optionally returns a value.

exit() / quit()

- Ends the program (usually used in scripts or interactive mode).

Assignment

- Used to store a value in a variable.

`A = 10`

`A = B`

`A += 10`

I/O

print

```
print("Ciao mondo!")
print(a,b)
print("Riga 1\nRiga 2\nRiga 3")
print("A", "B", "C", sep="-") # A-B-C
print("a", "Fine", end="!")  # Fine!
print("A",end=""); print("B") #AB
print("A", "B", "C", sep="-", end="?")
                                # A-B-C?
```

input

- In Python 3, the input() function allows you to read a string from standard input (keyboard).
- If you want to acquire an integer from the keyboard, you need to convert the string returned by input() using the conversion function int().
- If the user types a string that does not represent a number, a runtime error is generated

```
>>> x = input("Digita un numero e premi ENTER: ")
>>> y = int(input("Digita un numero e premi ENTER: "))
```

What is Formatted Printing?

Using f-strings (Python 3.6+)

```
name = "Alice"  
age = 25  
print(f"My name is {name} and I am {age} years old.")
```

Using format()

```
print("My name is {} and I am {} years old.".format(name, age))
```

%operator

```
x, y = 10, 20  
print("x = %d, y = %d" % (x, y))
```



Control FLOW

Control flow tools: WHILE

- While loops have the following general structure.

```
while expression:  
    statements
```

- Here, *statements* refers to one or more lines of Python code. The conditional expression may be any expression, where any non-zero value is true. The loop iterates while the expression is true.
- Note: All the statements indented by the same amount after a programming construct are considered to be part of a single block of code.

```
i = 1  
while i < 4:  
    print (i)  
    i = i + 1  
flag = True  
while flag and i < 8:  
    print (flag, i)  
    i = i + 1
```

```
1  
2  
3  
True 4  
True 5  
True 6  
True 7
```


Using indentation

```
a = 1  
if a:  
    print ("a is true!")
```

Control flow tools (IF)

- The if statement has the following general form.

```
if expression:  
    statements
```

- If the boolean expression evaluates to True, the statements are executed. Otherwise, they are skipped entirely.

```
a = 1  
b = 0  
if a:  
    print ("a is true!")  
if not b:  
    print ("b is false!")  
if a and b:  
    print ("a and b are true!")  
if a or b:  
    print ("a or b is true!")
```

```
a is true!  
b is false!  
a or b is true!
```

Control flow tools (IF ..ELSE)

- You can also pair an else with an if statement.

```
if expression:
    statements
else:
    statements
```

.

- Furthermore, if statements may be nested within each other.

```
a = 10
b = 0
c = 2
if a > b:
    if a > c:
        print ("a is greatest")
    else:
        print ("c is greatest")
```

a is greatest

Control flow tools (IF ..ELSE ... ELIF)

- You can also pair an else with an if statement.

```
if expression:
    statements
else:
    statements
elif expression:
    statements
```

- The elif keyword can be used to specify an else if statement.

```
a = 1
b = 0
c = 2
if a > b:
    if a > c:
        print ("a is greatest")
    else:
        print ("c is greatest")
elif b > c:
    print ("b is greatest")
else:
    print ("c is greatest")
```

c is greatest

Switch??

- In Python, there is no switch statement, but there are two methods to handle pattern matching:
- **Dictionary Mapping**
- **match-case** (Python 3.10+)

What is Dictionary Mapping?

- A Pythonic alternative to switch statements.
- Uses a dictionary where **keys represent cases** and **values represent actions or results**.

Pros

- **Fast lookup** ($O(1)$ time complexity).
- **Clean and readable** for many cases.
- Easy to **extend** and maintain.
- Can store **functions**, not just values.

Cons

- No built-in `default` keyword (must use `.get()` or handle manually).
- Limited to **exact matches** (cannot handle ranges or complex conditions easily).
- Slightly less intuitive for beginners compared to `if-elif`.

```
def switch_case(value):  
    actions = {  
        1: "One",  
        2: "Two",  
        3: "Three"  
    }  
    return actions.get(value, "Other")  
  
print(switch_case(2))  # Output: Two
```

```
def case_one(): return "Case 1"  
def case_two(): return "Case 2"  
  
switch = {1: case_one, 2: case_two}  
print(switch.get(1, lambda: "Default")())  
# Output: Case 1
```

What is match-case?

- Introduced in **Python 3.10** as **structural pattern matching**.
- Similar to switch in other languages, but **more powerful**.
- Matches **values** and **patterns** (including complex data structures)

```
value = 2

match value:
    case 1:
        print("One")
    case 2:
        print("Two")
    case 3:
        print("Three")
    case _:
        print("Other")    # Default case
```



```
point = (0, 5)

match point:
    case (0, 0):
        print("Origin")
    case (x, 0):
        print(f"X-axis at {x}")
    case (0, y):
        print(f"Y-axis at {y}")
    case (x, y):
        print(f"Point at ({x}, {y})")
```

pro

- Cleaner than if-elif chains.
- Handles **complex patterns** (tuples, lists, dicts).
- Supports **guards** (conditions):

```
match value:  
    case x if x > 10:  
        print("Greater than 10")
```

Control flow tools

The for loop has the following general form.

If a sequence contains an expression list, it is evaluated first. Then, the first item in the sequence is assigned to the iterating variable *var*. Next, the statements are executed. Each item in the sequence is assigned to *var*, and the statements are executed until the entire sequence is exhausted.

- For loops may be nested with other control flow tools such as while loops and if statements.

```
for var in sequence:  
    statements
```

```
from __future__ import print_function  
for letter in "aeiou":  
    print ("vowel: ", letter)  
for i in [1,2,3]:  
    print (i)  
for i in range(0,3):  
    print (i)
```

```
vowel: a  
vowel: e  
vowel: i  
vowel: o  
vowel: u  
1  
2  
3  
0  
1  
2
```

Control flow tools: range

range(start, stop, step):

You can specify the start, stop, and step values.start:

1. the first value in the range (default is 0).
2. stop: the range will stop just before this value.
3. step: the increment (default is 1).

```
from __future__ import print_function
for i in range(0, 4):
    print (i)
for i in range(0,8,2):
    print (i)
for i in range(20,14,-2):
    print (i)
```

0
1
2
3
0
2
4
6
20
18
16

Control flow tools

- `range` in Python 3 returns an iterable that generates numbers on demand (lazy evaluation), rather than creating a list of numbers in memory all at once, which is what `range` did in Python 2.

Control flow tools

There are four statements provided for manipulating loop structures: break, continue, pass, and else:

- **break**: terminates the current loop.
- **continue**: immediately begin the next iteration of the loop.
- **pass**: do nothing. Use when a statement is required syntactically.
- **else**: represents a set of statements that should execute when a loop terminates.

```
for num in range(10,20):  
    if num%2 == 0:  
        continue  
    for i in range(3,num):  
        if num%i == 0:  
            break  
    else:  
        print (num, 'is a prime number')
```

```
11 is a prime number  
13 is a prime number  
17 is a prime number  
19 is a prime number
```

A Solution Using basic python: Fibonacci sequence

```
from __future__ import print_function

total = 0
f1, f2 = 1, 2
while f1 < 4000000:
    if f1 % 2 == 0:
        total = total + f1
    f1, f2 = f2, f1 + f2
print(total)
```

Output: 4613732

Python supports multiple assignment at once. Right hand side is fully evaluated before setting the variables.